



Student Information System (SIS)

Software Design and Architecture (SDA) — Assignment 1: Object-Oriented Class Design

Student Name: Sude Hatkaoğlu

Course: Software Design and Architecture (SDA) / Object Oriented Design

Instructor: Prof. Dr. Ali Güneş

Submission Target: GitHub Repository & Term Project Registration

Technology Stack: Java 21+ / 25 (JavaBean Specification, Pure OOP Architecture)

1. Executive Summary & Objective

This document details the architectural design, object-oriented principles, and implementation specifics of the **Student Information System (SIS)**. The system is modeled strictly against the course relational database schema (ER Diagram) and demonstrates modern object-oriented programming standards in Java.

The system encompasses all 9 database entities, models foreign-key references via object aggregation and type-safe UUID identifiers, leverages inheritance for common person attributes, and validates operation through an integrated registration and curriculum simulation.

2. Object-Oriented Design Principles Applied

- **Encapsulation:** All model class attributes are strictly declared as `private`. Access is mediated through public getter and setter methods adhering to standard JavaBean naming conventions.
- **Inheritance (Kalıtım):** Common identity and contact fields (`id`, `nationalId`, `firstName`, `lastName`, `email`, `phone`) are abstracted into an abstract superclass `BasePerson`, inherited by both `Student` and `Instructor`.
- **Object Aggregation (FK Simulation):** Rather than maintaining raw foreign-key identifiers, classes hold direct object references (e.g., `Student` references `Program`; `Program` references `Department`; `Department` references `Faculty`).
- **Type Safety with Enums:** All domain-restricted categorical fields (`Gender`, `DegreeLevel`, `StudentStatus`, `AcademicTitle`, `CourseType`, `PrerequisiteType`, `Semester`) are implemented as standalone Java enum types.
- **Circular Reference Prevention:** Custom `toString()` overrides format related object summaries (e.g., department name, dean title) without invoking recursive representations, avoiding potential `StackOverflowError`.

3. Database Schema to Java Model Mapping

Database Table	Java Class	Inheritance / FK References	Enum Fields
(Abstract Base)	BasePerson	Root entity for individuals	-
FACULTIES	Faculty	dean → Instructor	-
DEPARTMENTS	Department	faculty → Faculty , headInstructor → Instructor	-
PROGRAMS	Program	department → Department	DegreeLevel
INSTRUCTORS	Instructor	extends BasePerson , department → Department	AcademicTitle
STUDENTS	Student	extends BasePerson , program → Program	Gender (E, K) , StudentStatus
COURSES	Course	department → Department	CourseType
COURSE_PREREQUISITES	CoursePrerequisite	course → Course , prerequisiteCourse → Course	PrerequisiteType
ACADEMIC_TERMS	AcademicTerm	-	Semester
PROGRAM_COURSES	ProgramCourse	program → Program , course → Course	CourseType

4. Compilation, Execution & Output

The project compiles with zero warnings under modern Java compilers and executes directly:

```
# Compilation:
javac -encoding UTF-8 -d out src/*.java

# Execution:
java -cp out Main
```

Simulation Output Log:

```
=== ÖĞRENCİ BİLGİ SİSTEMİ SİMÜLASYONU ===

Aktif Dönem: 2026-2027 Güz Dönemi (2026-2027 / Güz)
Ön Koşul Kuralı: SWE201 dersi için ön koşul -> SWE101 (Min Not: DD)

Program Müfredatı (ProgramCourse):
  * SWE-BS | Dönem: 1 -> Programlamaya Giriş
  * SWE-BS | Dönem: 3 -> Nesne Yönelimli Tasarım

--- Kayıt Edilen Öğrenci Listesi ---
```

Öğrenci: Sude Hatkaoglu (2026101001) | Program: Yazılım Mühendisliği Lisans Programı | Sınıf: 1 | Durum:

Aktif

* Cinsiyet : Kadın
* Bağlı Olduğu Fakülte : Mühendislik Fakültesi
* Fakülte Dekanı : Prof. Dr. Ali Güneş
* Kayıt Tarihi : 2026-10-01T...

Document generated for Software Design and Architecture — Assignment 1 © 2026 Sude Hatkaoglu